Ex. No: 3 System Calls: Fork, Exit, Getpid, Wait, Close

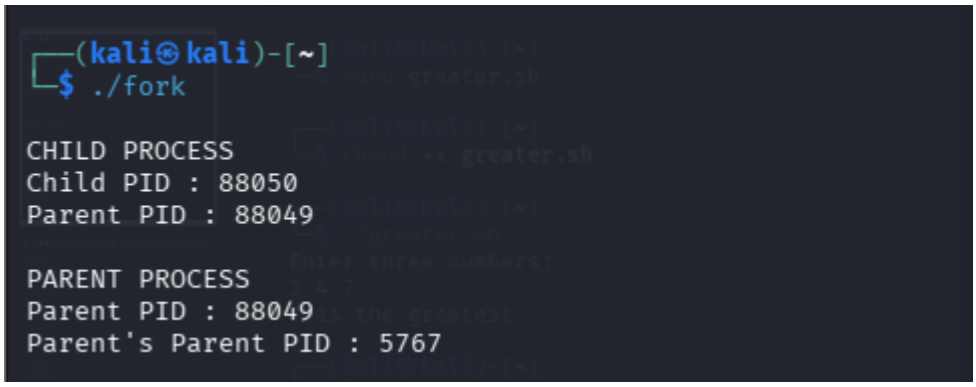
PROGRAM 1: FORK(), GETPID(), WAIT(), EXIT()

C PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
pid_t pid;
pid = fork();
if(pid < 0)
{
printf("Fork Failed\n");
exit(1);
}
else if(pid == 0)
{
printf("\nCHILD PROCESS");
printf("\nChild PID : %d", getpid());
printf("\nParent PID : %d\n", getppid());
exit(0);
}
else
{
wait(NULL);
printf("\nPARENT PROCESS");
printf("\nParent PID : %d", getpid());
printf("\nParent's Parent PID : %d\n", getppid());
}
return 0;
```

}

Output:



SHELL SCRIPT:

#!/bin/bash

echo "Parent Process ID : $$"

(

echo "Child Process ID : $$"

echo "Parent Process ID : $PPID"

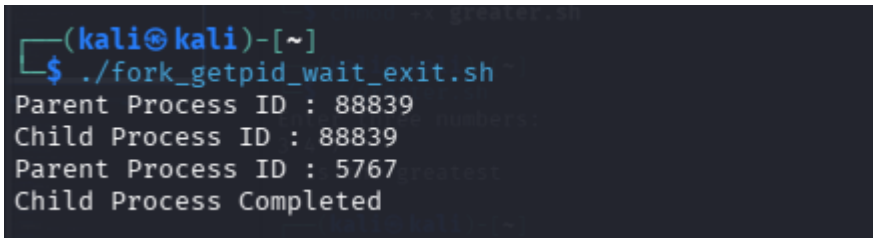
exit 0

)&

wait

echo "Child Process Completed"

Output:



PROGRAM 2: WAIT() SYSTEM CALL

C PROGRAM

#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

int main() {

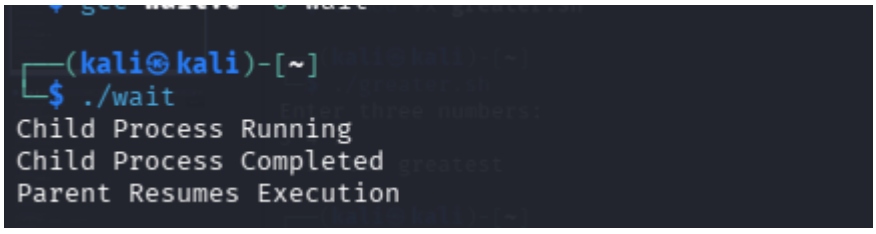
```
pid_t pid;
pid = fork();
if(pid == 0)
{
printf("Child Process Running
\n");

sleep(5);
printf("Child Process Completed
\n");

}
else
{
wait(NULL);
printf("Parent Resumes Execution
\n");

}
return 0;
}
```

Output:

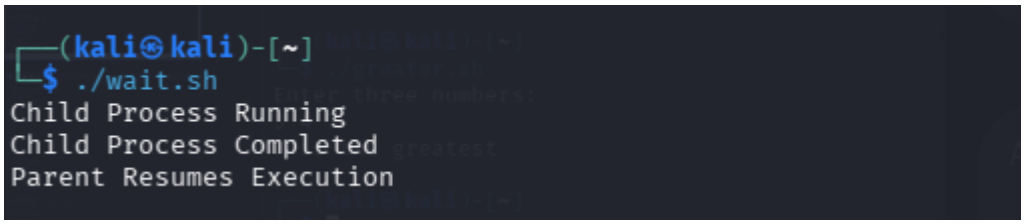


SHELL SCRIPT

```
#!/bin/bash (
echo "Child Process Running"
sleep 5
```

```
echo "Child Process Completed"
) &
wait
echo "Parent Resumes Execution"
```

Output:

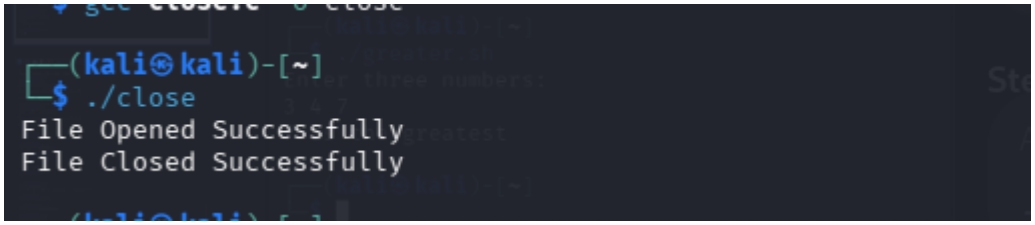


PROGRAM 3: CLOSE() SYSTEM CALL

C PROGRAM

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main()
{
int fd;
fd = open("sample.txt", O_RDONLY);
if(fd < 0)
{
printf("File Opening Failed\n");
return 1;
}
printf("File Opened Successfully\n");
close(fd);
printf("File Closed Successfully\n");
return 0;
}
```

Output:



SHELL SCRIPT

Shell scripting does not provide a direct close() system call. File descriptors can be opened and closed

as follows:

#!/bin/bash

exec 3< sample.txt

echo "File Opened Successfully"

exec 3<&-

echo "File Closed Successfully"

Output: